

Chapter 3

Finite State Machine (FSM) and Morphology

Overview 6hrs

1. Introduction to FSM and FST
2. Morphological processes
3. Principle of word construction (Suffix, Prefix, Stem, Affixes): English, Nepali, Turkish
4. Morphological Representation and FSM: Lexicon, Morphotactics and Orthographic Rules
5. Morphological parsing and FST
6. Mealy Machine
7. FST operations

Formal Languages

An alphabet Σ is a set of symbols:

e.g. $\Sigma = \{a, b, c\}$

A string ω is a sequence of symbols, e.g. $\omega = abcb$.

The empty string ε consists of zero symbols.

The Kleene closure Σ^* ('sigma star') is the (infinite) set of all strings that can be formed from Σ :

$\Sigma^* = \{\varepsilon, a, b, c, aa, ab, ba, aaa, \dots\}$

Automata

An **automaton** is an abstract model of a computer.
It **reads** an input string symbol by symbol.
It **changes** its internal state depending on
the **current input symbol** and its **current internal state**.

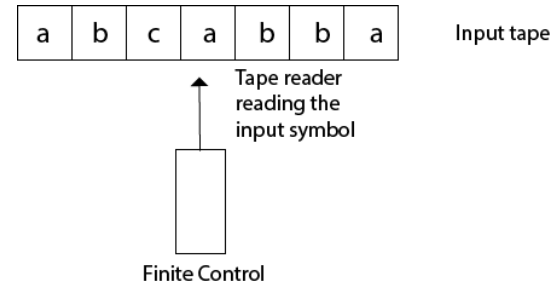
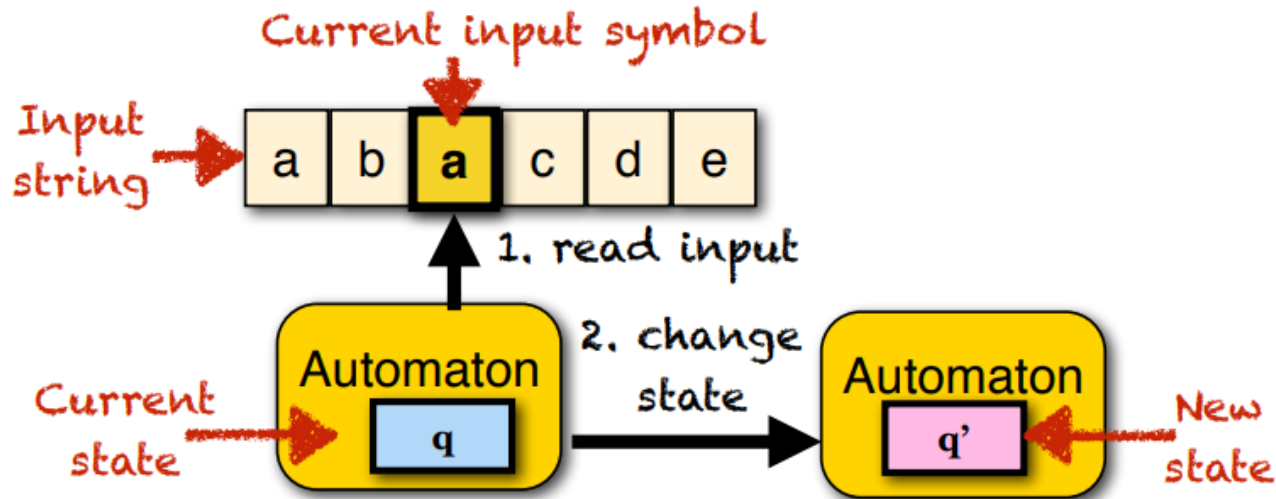
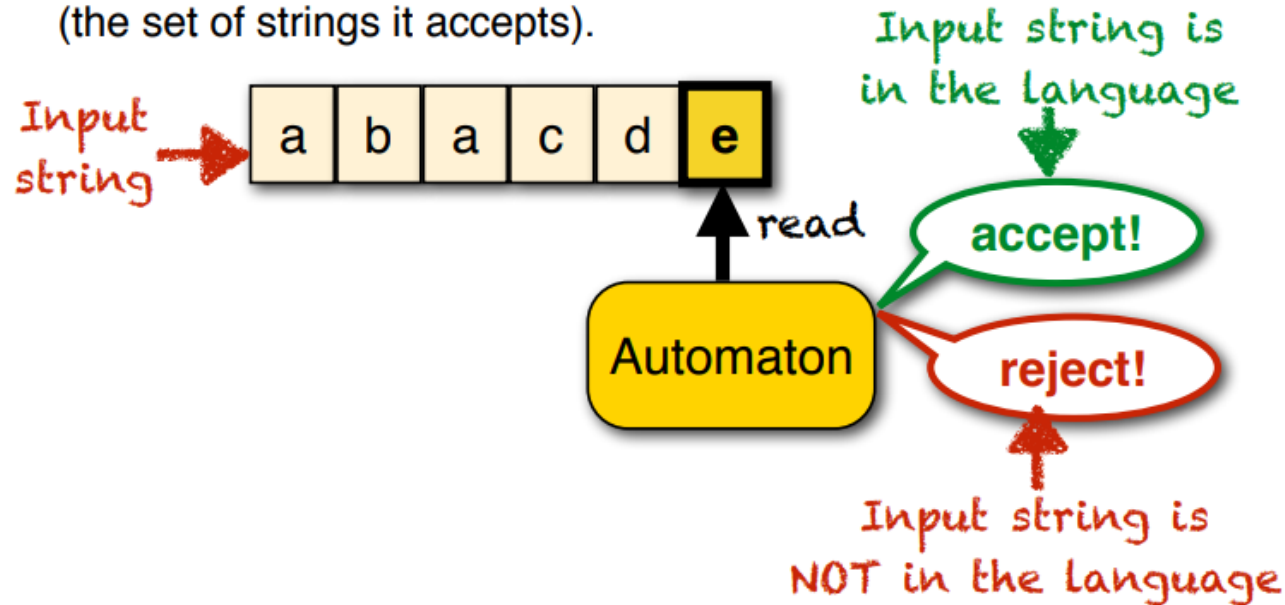


Fig :- Finite automata model

Automata

The automaton either **accepts** or **rejects** the input string.

Every automaton defines a language
(the set of strings it accepts).



Automata

Different types of automata define different language classes:

- **Finite-state** automata define **regular** languages
- **Pushdown** automata define **context-free** languages

Introduction to Finite State Machine (FSM)

- Finite state machine **is used to recognize patterns.**
- Finite automata machine **takes the string of symbol as input** and **changes its state** accordingly. In the input, when a desired symbol is found then the transition occurs.
- While transition, the automata can **either move to the next state** or **stay in the same state.**
- FA has two states: **accept state** or **reject state**. When the input string is successfully processed and the automata reached its final state then it will accept.
- There are two types of finite automata:
 - 1. DFA (deterministic finite automata)
 - 2. NFA (non-deterministic finite automata)

FSM Model

- Finite automata can be represented by input tape and finite control.
 - **Input tape:** It is a linear tape having some number of cells. Each input symbol is placed in each cell.
 - **Finite control:** The finite control decides the next state on receiving particular input from input tape. The tape reader reads the cells one by one from left to right, and at a time only one input symbol is read.
- The automation either accepts or rejects the input string.
- If it reads all the input string and reaches the final state, it accepts that string
- Otherwise not.

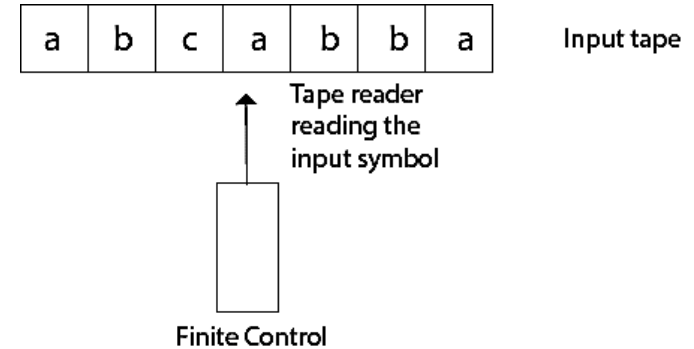


Fig :- Finite automata model

Formal Definition of FA

A finite automaton is a collection of 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where:

- Q : finite set of states
- Σ : finite set of the input symbol
- q_0 : initial state
- F : final state
- δ : Transition function

Deterministic Finite Automation (DFA)

A DFA can be represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where –

- Q is a finite set of states.
- Σ is a finite set of symbols called the alphabet.
- δ is the transition function where $\delta: Q \times \Sigma \rightarrow Q$
- q_0 is the initial state from where any input is processed ($q_0 \in Q$).
- F is a set of final state/states of Q ($F \subseteq Q$).

Deterministic Finite Automaton (DFA): Example

Let a deterministic finite automaton be $\rightarrow$

$Q = \{a, b, c\}$,

$\Sigma = \{0, 1\}$,

$q_0 = \{a\}$,

$F = \{c\}$, and

Transition function δ as shown by the following table –

Q	0	1
a	a	b
b	c	a
c	b	c

Or the transition function can be written as:

$\delta(a, 0) \rightarrow a$

$\delta(a, 1) \rightarrow b$

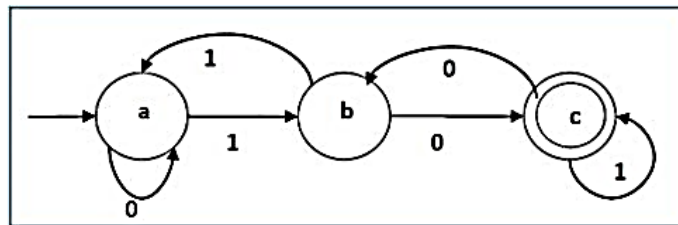
$\delta(b, 0) \rightarrow c$

$\delta(b, 1) \rightarrow a$

$\delta(c, 0) \rightarrow b$

$\delta(c, 1) \rightarrow c$

Its graphical representation would be as follows –



Numerical:

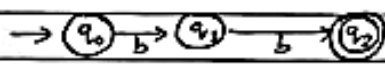
1. Design a DFA that accepts the language given by: $L = \{w \in \{a, b\}^* : w \text{ has even numbers of } b\}$

Numerical:

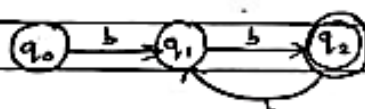
1. Design a DFA that accepts the language given by: $L = \{w \in \{a, b\}^* : w \text{ has even numbers of } b\}$

Solⁿ: Here, $L = \{abb, bb, bbbb, bab, bba, bbaab, baababb, \dots\}$

First let us design for bb



Again design for $bbbb$



Hence the DFA is

$$M = \{Q, \Sigma, \delta, q_0, f\}$$

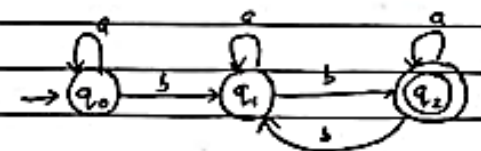
where,

$$Q = \{q_0, q_1, q_2\}$$
$$\Sigma = \{a, b\}$$
$$q_0 = \{q_0\}$$
$$F = \{q_2\}$$

δ :

Q	a	b
q_0	q_0	q_1
q_1	q_1	q_2
q_2	q_2	q_1

State diagram is:



2. Design a DFA that accepts the language given by: $L = \{w \in \{1, 0\}^* : w \text{ ends with } 100\}$

Here,
 $L = \{100, 1100, 0100, 10100, 010100, \dots\}$

Let $M = \{Q, \Sigma, \delta, q_0, F\}$ where

$Q = \{q_0, q_1, q_2, q_3\}$

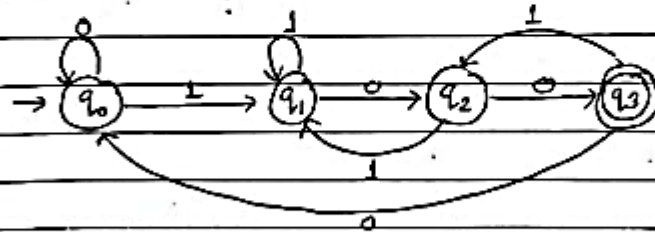
$\Sigma = \{0, 1\}$

$q_0 = \{q_0\}$

$F = \{q_3\}$

$\delta:$	Q	0	1
	q_0	q_0	q_1
	q_1	q_2	q_1
	q_2	q_3	q_1
	q_3	q_0	q_1

Hence, the state diagram is:



Verification:

10100 → Verified ends at q_3 , accepted

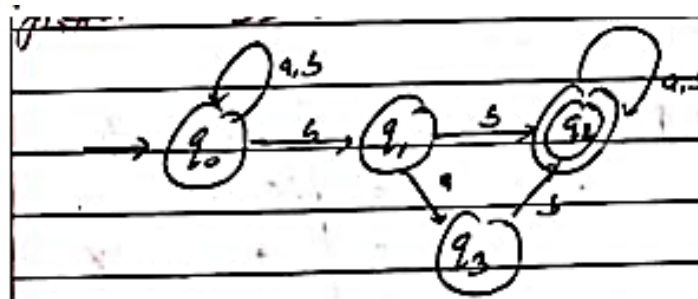
10101 → ends at q_1 , rejected

NDFA

An NDFA can be represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where –

- Q is a finite set of states.
- Σ is a finite set of symbols called the alphabets.
- δ is the transition function where $\delta: Q \times \Sigma \rightarrow 2^Q$
(Here the power set of Q (2^Q) has been taken because in case of NDFA, from a state, transition can occur to any combination of Q states)
- q_0 is the initial state from where any input is processed ($q_0 \in Q$).
- F is a set of final state/states of Q ($F \subseteq Q$).

1. Design a NFA that accepts the set of strings containing occurrence of pattern bb or bab



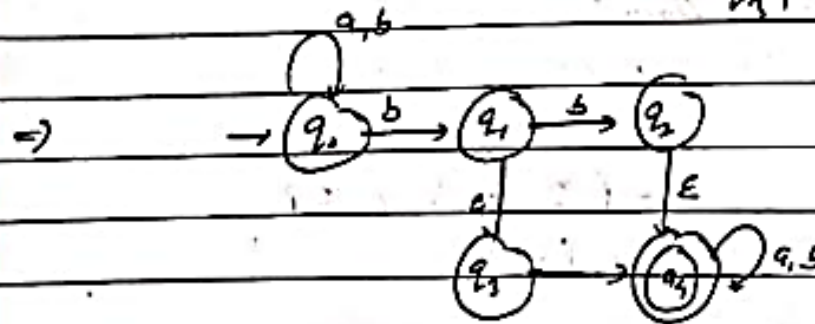
Here, $Q = \{q_0, q_1, q_2, q_3\}$

$\Sigma = \{a, b\}$

$q_0 = \{q_0\}$

$F = \{q_2\}$

$\delta:$	Q	a	b
q_0	q_0	$\{q_0, q_1\}$	
q_1	q_2	q_1	q_2
q_2	q_2	q_2	q_2
q_3	$\emptyset$	q_1	q_2



Assignment:

1. Construct a DFA starting with aba, $\Sigma = a, b$.
2. Design a DFA that accepts the language given by: $L = \{w \in \{a, b\}^* : w \text{ does not contain three consecutive } b\}$
3. Design a DFA that accepts the language given by: $L = \{w \in \{0, 1\}^* : w \text{ starts with } 01 \text{ having even length}\}$
4. Design a DFA that accepts the language given by: $L = \{w \in \{0, 1\}^* : w \text{ starts with } 01\}$

State Diagrams and State Tables

1. At the start of a design of a machine, the total number of states required are determined.
2. This is achieved by drawing a state diagram, which shows the internal states and the transitions between them.
3. The transition can be represented by state table.

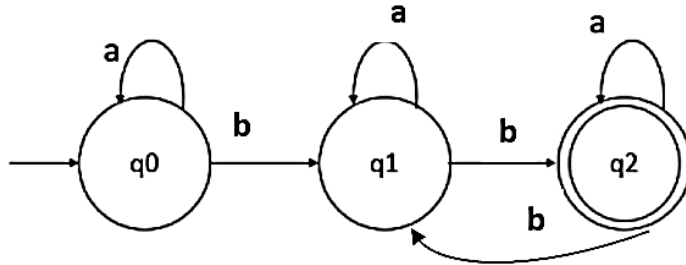


Figure: State Diagram

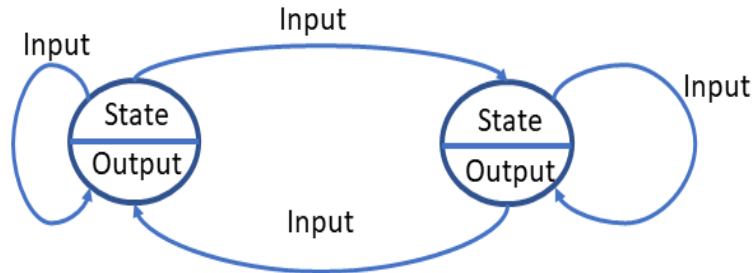
Q	a	b
q ₀	q ₀	q ₁
q ₁	q ₁	q ₂
q ₂	q ₂	q ₁

Figure: State Table

Moore and

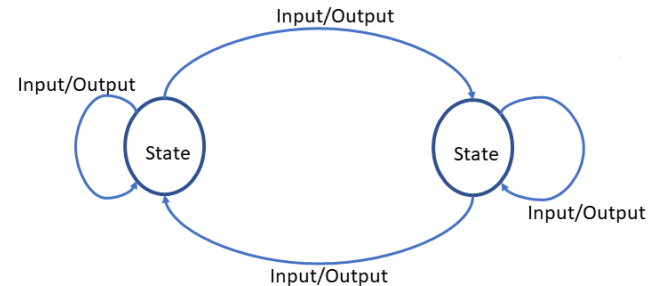
Mealy Machine

- In this state machine, the output is determined only by the present active state of the state machine and not by any input events.
- Output is represented inside the state.



Generic Moore model

- In this machine, the output produced by the state machine depends on the input events fed to the state machine AND presents an active state of the state machine.
- The output is not produced inside the state.

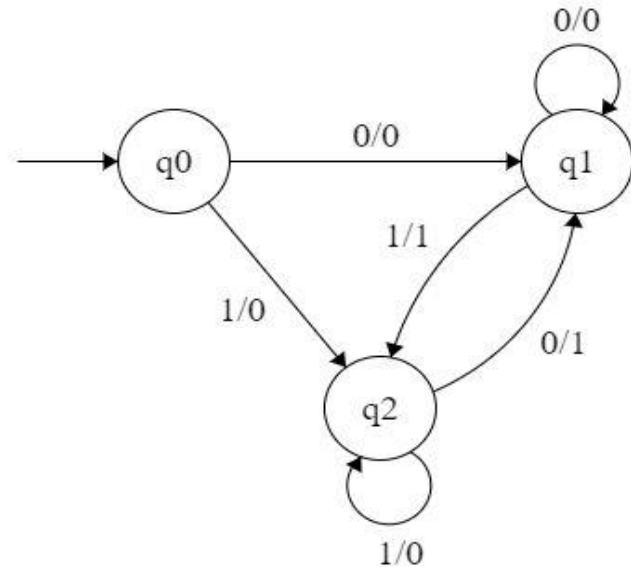


Generic Mealy model

Mealy Machine Example

- In the mealy machine shown in Figure , the output is represented with each input symbol for each state separated by /.
- The length of output for a mealy machine is equal to the length of input.
- **Input:** 11
- **Transition:** $\delta(q0, \underline{1}) \Rightarrow \delta(q2, \underline{1}) \Rightarrow q2$
- **Output:** 00

(q0 to q2 transition has Output 0 and q2 to q2 transition also has Output 0)



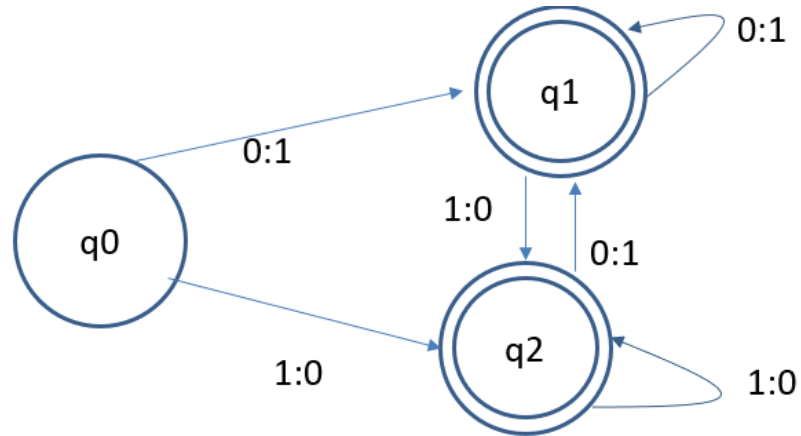
Classwork:

1. Construct a Mealy machine to generate 1's complement of a given binary number.

Basic idea: If we apply input 1, and then its output will be 0, and if input is 0 then output will be 1.

So, we design the machine to test for below strings:

0 , 1 , 00, 11, 01, 10, 01, 100, 001, 101, 1001, 1010, and so on.



Principle of word construction (Suffix, Prefix, Stem, Affixes): English, Nepali

Root words and Base words

- Root Words and Base Words are the same.
- Root words and base words are the most basic part of a word.
- If you add an affix to a root or base word you change the meaning of the word.



"Cook" is a base word.

Affixes

- Affixes are word parts that change the meaning of a root or base word.
- Prefixes and Suffixes are both Affixes.



un+cook+ed=uncooked

Affix → ↑
 Affix

Prefixes

- **Prefixes** are word parts (affixes) that comes at the **beginning** of the root word or base word.



Diagram illustrating the formation of the word "uncooked":

un + cook + ed = uncooked

The prefix "un" is highlighted in red, and the suffix "ed" is highlighted in blue. An arrow points to "un" with the label "Prefix", and another arrow points to "ed" with the label "Affix".

- Prefix: **im-**
Meaning: **not**
- Examples:
- 

impossible
- 

impolite

Suffix

- **Suffixes** are word parts (affixes) that come at the end of the root word or base word.



un+cook+ed=uncooked

Prefix →

↑ Suffix

- Suffix: **er**
- Meaning: **one who takes part in**
- Examples:
- 

teacher
- 

driver

Most common prefixes and suffixes in English Language

Prefix	Meaning	Key Word
anti-	against	antifreeze
de-	opposite	defrost
dis-*	not, opposite of	disagree
en-, em-	cause to	encode, embrace
fore-	before	forecast
in-, im-	in	infield
in-, im-, il-, ir-*	not	injustice, impossible
inter-	between	interact
mid-	middle	midway
mis-	wrongly	misfire
non-	not	nonsense
over-	over	overlook
pre-	before	prefix
re-*	again	return
semi-	half	semicircle
sub-	under	submarine
super-	above	superstar
trans-	across	transport
un-*	not	unfriendly
under-	under	undersea

Suffix	Meaning	Key Word
-able, -ible	can be done	comfortable
-al, -ial	having characteristics of	personal
-ed*	past-tense verbs	hopped
-en	made of	wooden
-er	comparative	higher
-er,	one who	worker, actor
-est	comparative	biggest
-ful	full of	careful
-ic	having characteristics of	linguistic
-ing*	verb form/ present participle	running
-ion, -tion, -ation, ition	act, process	occasion, attraction
-ity, -ty	state of	infinity
-ive, -ative, -itive	adjective form of a noun	plaintive
-less	without	fearless
-ly*	characteristic of	quickly
-ment	action or process	enjoyment
-ness	state of, condition of	kindness
-ous, -eous, -ious	possessing the qualities of	joyous
-s, -es*	more than one	books, boxes
-y	characterized by	happy

Morphology

- Morphology is the study of the way words are built from smaller meaningful units called **morphemes**.
- We can divide morphemes into two broad classes.
 - **Stems** – the core meaningful units, the root of the word.
 - **Affixes** – add additional meanings and grammatical functions to words.
- Affixes are further divided into:
 - **Prefixes** – precede the stem: do / undo
 - **Suffixes** – follow the stem: eat / eats
 - **Infixes** – are inserted inside the stem: spoonful/ spoonsful , passerby/passersby
 - **Circumfixes** – precede and follow the stem : enlighten, enlargen
- English doesn't stack more affixes.
- But Turkish can have words with a lot of suffixes.
- Languages, such as Turkish, tend to string affixes together are called **agglutinative** languages.

Inflectional and Derivational Morphology

There are two broad classes of morphology:

- **Inflectional morphology**
- **Derivational morphology**

a) Inflectional Morpheme

- After a combination with an **inflectional morpheme**, the meaning and class of the actual stem usually do not change.
 - eat / eats pencil / pencils
 - gel / geliyorum masa / masam

b) Derivational Morpheme

- After a combination with an **derivational morpheme**, the meaning and the class of the actual stem usually change.
 - compute / computer do / undo friend / friendly
 - Uygar / uygarlaş kapı / kapıcı
- The irregular changes may happen with derivational affixes.

a) English Inflectional Morphology

- Nouns have simple inflectional morphology.
 - plural -- cat / cats
 - possessive -- John / John's
- Verbs have slightly more complex inflectional, but still relatively simple inflectional morphology.
 - past form -- walk / walked
 - past participle form -- walk / walked
 - gerund -- walk / walking
 - singular third person -- walk / walks
- Verbs can be categorized as:
 - main verbs
 - modal verbs -- can, will, should
 - primary verbs -- be, have, do
- Regular and irregular verbs: walk / walked -- go / went
- **Adjectives:**
 - There is the base form like OLD, the comparative form: OLDER and the superlative form: OLDEST.
- **Adverbs:**
 - There is the base form like SOON, the comparative form: SOONER and the superlative form: SOONEST

b) English Derivational Morphology

- Some English derivational affixes
 - -ation : transport / transportation
 - -er : kill / killer
 - -ness : fuzzy / fuzziness
 - -al : computation / computational
 - -able : break / breakable
 - -less : help / helpless
 - un : do / undo
 - re : try / retry

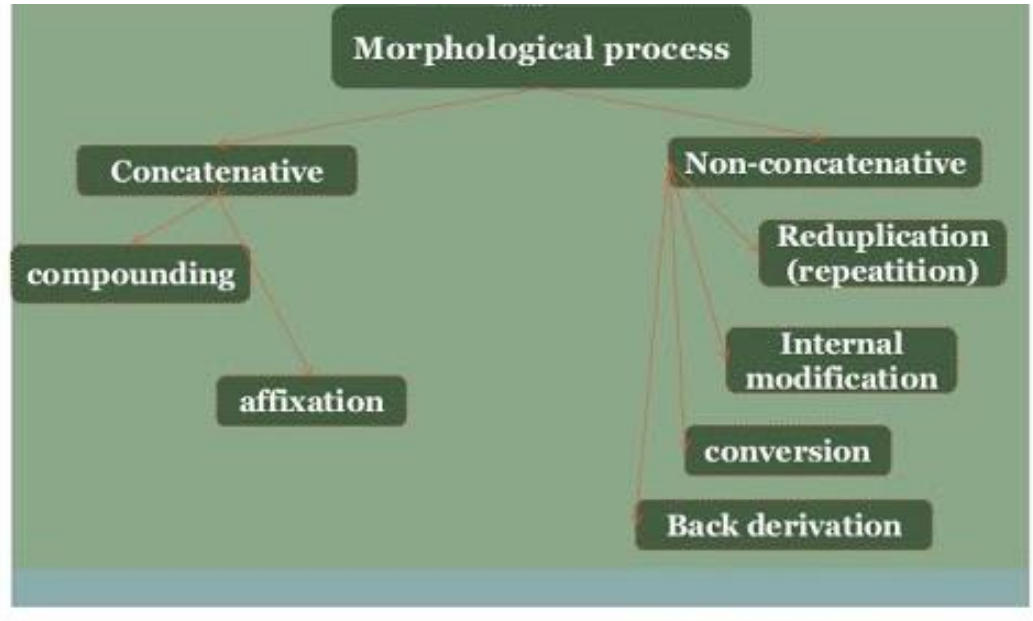
Morphological processes

- A **morphological process** is a linguistic process that creates word forms by applying the morphological rules within a morphological system.
- The morphological process is the process by which a word is adjusted to conform to a certain context.
- To put it simply, it is the **process of changing the form and function of a word to fit a context**, sometimes to the extent of changing the meaning and/or grammatical function.
- These processes are not just used in the English language, other languages use it too.
- ***Morphological process is a mean of changing a stem to adjust its meaning to fit its syntactic and communicational context.***

Reference; <https://www.theedadvocate.org/what-to-know-about-morphological-processes/>

Morphological processes

- Two ways of morphological process:
 1. **Concatenative**: putting morphemes together
 2. **Non-concatenative**: modifying internal structure of morphemes



1. COMPOUNDING

- English shares with many languages the ability to create new words by combining old words.
- compounding can be analyzed through its constituents.



- Open Compounds:** Compounds written as separate words.
e.g : end zone, high school.
- Closed Compounds:** Compounds written as single words.
e.g : newspaper, goldfish, highway.
- Hyphenated Compounds:** Compounds that are hyphenated.
e.g : mother-in-law, second-rate.

2. AFFIXATION

- Prefixes are letters that are added to the *beginning* of a word.
- A prefix changes the meaning of a word.

Example:

"preschool"

- The prefix is *pro-*
- The prefix *pre* means "before" so the word *presch* means "before school"

Negative and Positive	Size	Location	Time and Order	Number
Un-	Semi-	Inter-	Pre-	Mono-
Non-	Mini-	Super-	ante-	Bi-
In-	Micro-	Trans-	Fore-	Hex-
Dis-		Ex-	Post-	Oct-
Re-		Extra-		Multi-

- ❑ Suffixes are letters added to the *end* of a word.
- ❑ A suffix also changes the meaning and the word class of a word.

Example:

careless

The suffix is **-less**

The suffix **-less** means **without** so the word **careless** means "without care"

3. REDUPLICATION (REPETATION)

This process can be classified according to the amount of form that is duplicated, whether complete or partial, and it the letter according to exactly which part.

SEVERAL TYPES OF REDUPLICATION IN ENGLISH

- ❑ **Rhyming reduplication:** hokey-pokey, razzle-dazzle, super-duper, boogie-woogie, teenie-weenie, walkie-talkie
- ❑ **Exact reduplications (baby-talk-like):** bye-bye, choo-choo, night-night, no-no, pee-pee, poo-poo.
- ❑ **Ablaut reduplications:** bric-brac, chit-chat, criss-cross, kitty-cat, knick-knack, pitter-patter, splish-splash, zig-zag.

4. INTERNAL MODIFICATION

a. Vowel modification

b. Consonant modification

c. Mixed modification

d. Tonal modification

e. Stress modification

f. suppletion

a. Vowel modification

verbs in English:

- [ɪ] – [œ] *begin – began, ring – rang, sing – sang,*
- [i:] – [ou] *speak – spoke, steal – stole,*
- [ai] – [au] *bind – bound, find – found,*

B. CONSONANT MODIFICATION

• Examples:

• Noun/Verb in English:

- [f] – [v] *belief – believe, grief – grieve, proof – prove,*
- [s] – [z] *advice – advise, device – devise, use – use,*
- [s] – [d] *defense – defend, offence – offend,*
- [t] – [d] *bent – bend, ascent – ascend, descent – descend,*

c. Mixed modification

Examples:

English present/past: *catch – caught, seek – sought, teach – taught*

Verb / Noun: *live – life*

a. Vowel modification

verbs in English:

- ❑ [ɪ] – [oɛ] *begin – began, ring – rang, sing – sang,*
- ❑ [i:] – [ou] *speak – spoke, steal – stole,*
- ❑ [aɪ] – [au] *bind – bound, find – found,*

B. CONSONANT MODIFICATION

• Examples:

• Noun/Verb in English:

- ❑ [f] – [v] *belief – believe, grief – grieve, proof – prove,*
- ❑ [s] – [z] *advice – advise, device – devise, use – use,*
- ❑ [s] – [d] *defense – defend, offence – offend,*
- ❑ [t] – [d] *bent – bend, ascent – ascend, descent – descend,*

c. Mixed modification

Examples:

English present/past: *catch* – *caught*, *seek* – *sought*, *teach* – *taught*

Verb / Noun: *live* – *life*

D. TONAL MODIFICATION (TONE)

A NUMBER OF AFRICAN LANGUAGES
USE TONAL MODIFICATION FOR VERB
INFLECTION.

he saw'

→ Near Past : _ ^ ^ [a:Bo:ne]

→ Perfect : ^ \ _ [a:Bo:ne]

(where ^ = high tone, _ = low tone, \ = falling tone,
and B is an implosive bilabial stop)

E. STRESS MODIFICATION

A base can undergo a change in the placement of stress to reflect a change in its category.

Examples:

Noun

Primary stress on: First syllable

récord

cóntrast

súbject

Verb

Second syllable

re~~cor~~d

con~~tra~~st

sub~~je~~ct

f. Suppletion (total modification)

a morphological process whereby a root morpheme is replaced by a phonologically unrelated form in order to indicate a grammatical contrast.

Examples:

Basic form

I

be

good

Suppletive form

me

were

well

5. Conversion

A process by which a word belonging to one word class without any change of form but the function of word is change.

TYPES OF CONVERSION

- Verb to noun
to attack → attack
to print out → a printout
- Noun to verb
comb → to comb
chair → to chair
- Name to verb
Harpo → to Harpo
Houndini → to Houndini

- Adjective to verb
dirty → to dirty
slow → to slow
- Preposition to verb
out → to out

In some cases, conversion is accompanied by a change in the stress pattern known as **stress shift**

6. BACKFORMATION

A process in which a word changes its forms and function

Typically, a word of one type, which is usually a noun, is reduced to form a word of another type, usually a verb.

e.g

television (N) → televise (V)

donation (N) → donate (V)

Parts of A Morphological Processor

- For a morphological processor, we need at least followings:
 1. **Lexicon** : It is the **list of stems and affixes together** with basic information about them such as their main categories (noun, verb, adjective, ...) and their sub-categories (regular noun, irregular noun, ...).
 2. **Morphotactic** : The **model of morpheme ordering** that explains which classes of morphemes can follow other classes of morphemes inside a word.
 3. **Orthographic Rules (Spelling Rules)** : These spelling rules **are used to model changes that occur in a word** (normally when two morphemes combine).

1. Lexicon

- A lexicon is a **repository for words** (stems).
- They are **grouped according to their main categories**.
 - noun, verb, adjective, adverb, ...
- They may be also divided into sub-categories.
 - regular-nouns, irregular-singular nouns, irregular-plural nouns, ...
- The simplest way to create a morphological parser, put all possible words (together with its inflections) into a lexicon.

2. Morphotactic

- Which morphemes can follow which morphemes.
- **Morphotactic** is the **syntax of morphemes: what order they come in**, what kind of units they make.

Lexicon:

regular-noun

fox

cat

dog

irregular-pl-noun

geese

sheep

mice

irreg-sg-noun

goose

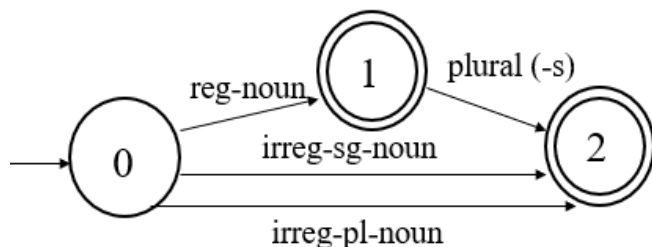
sheep

mouse

plural

-s

- Simple English Nominal Inflection (Morphotactic Rules)



Example:

(q0, fox) → q1

(q1, s) → q2

Here, **cat** and **cats** are accepted by the FSM

3. Orthographic Rules

- We need FSTs to map intermediate level to surface level.
- For each spelling rule we will have a FST, and these FSTs run parallel.
- Some of English Spelling Rules:
 - a) **consonant doubling:**
 - 1-letter consonant doubled before ing/ed
 - Example: beg/begging
 - b) **E deletion:**
 - Silent e dropped before ing and ed
 - Example: make/making
 - c) **E insertion:**
 - e added after s, z, x, ch, sh before s
 - Example: watch/watches
 - d) **Y replacement:**
 - y changes to ie before s, and to i before ed
 - Example: try/tries try/tried
 - e) **K insertion:**
 - verbs ending with vowel+c we add k
 - Example: panic/panicked

Finite-State Transducers

- In NLP, a Finite State Transducer (FST) is a computational model **used for representing and manipulating finite state machines (FSMs) that map input sequences to output sequences.**
- FSTs are widely used in various NLP tasks such as morphological analysis, spell checking, speech recognition, and machine translation.
- Formally, an FST can be represented as a 5-tuple

- $$T = (Q, \Sigma, \Delta, \delta, F)$$

- Where:

- Q= finite set of states

- Σ =set finite input alphabets

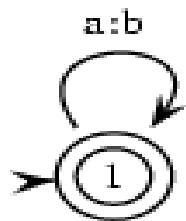
- Δ = finite output alphabet

- F= a set of final states

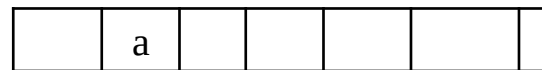
- δ = a set of transition function

Finite-State Transducers

- an augmentation of FSAs in which there are two tapes
 - A two-tape automaton, the upper tape an "underlying representation" tape and
 - the lower a "surface representation"

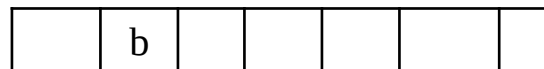


Lexical Tape



(upper tape)

Surface Tape



(lower tape)

a : b at the arc means that in this transition the transducer reads **a** from the first tape and writes **b** onto the second.

Two-Level Morphology with FST

- Two-level morphology represents the correspondence between lexical and surface levels.
- We use a finite-state transducer to find mapping between these two levels.
- A FST is a two-tape automaton:
 - Reads from one tape, and writes to other one.
- For morphological processing, one tape holds lexical representation, the second one holds the surface form of a word.

Lexical Tape

	d	o	g	+N	+PL	
--	---	---	---	----	-----	--

 (upper tape)

Surface Tape

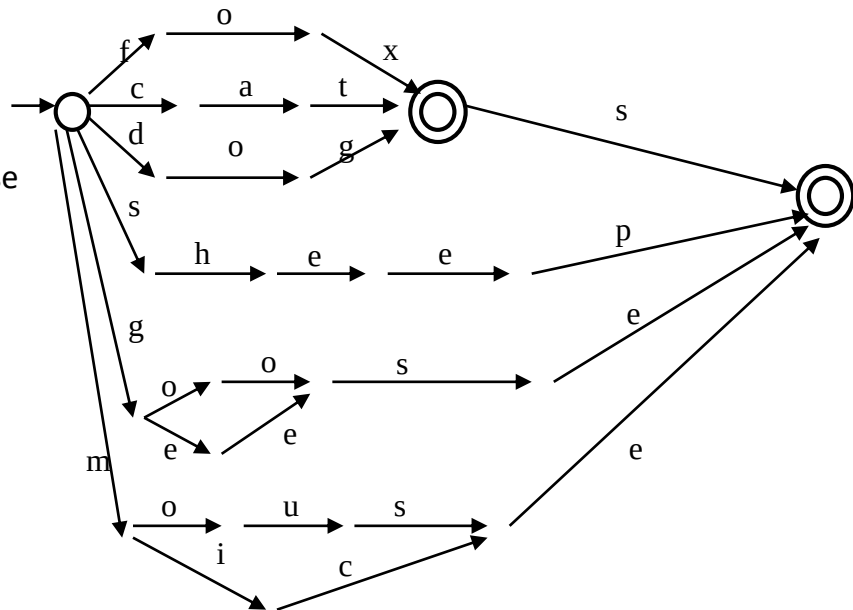
	d	o	g	s		
--	---	---	---	---	--	--

 (lower tape)

FST for stems

- Stemming is the process of reducing words to their root or base form, often by removing affixes such as prefixes and suffixes.
- FSTs can be used to perform stemming efficiently by defining rules for stripping affixes and producing the stem of a word.
- A FST for stems which maps roots to their root-class

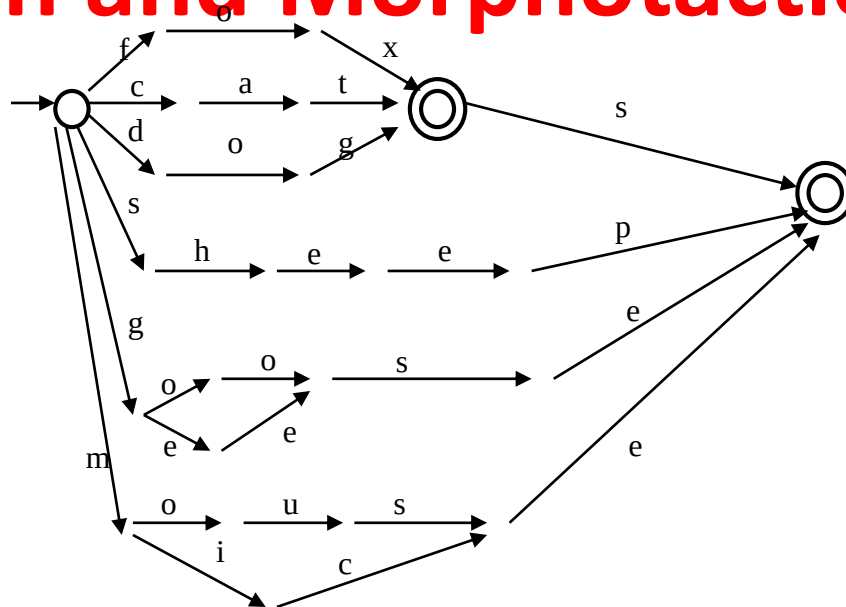
<u>reg-noun</u>	<u>irreg-pl-noun</u>	<u>irreg-sg-noun</u>
fox	g o:e o:e se	goose
cats	sheep	sheep
dog	m o:i u:ε s:c e	mouse



This only says yes or no. Does not give lexical representation.
It accepts a wrong word (foxs).

- fox** stands for **f:f o:o x:x**
- When these two transducers are composed, we have a FST which maps lexical forms to intermediate forms of words for simple English noun inflections.
- Next thing that we should handle is to design the FSTs for orthographic rules, and combine all these transducers.

Combine Lexicon and Morphotactics



This only says yes or no. Does not give lexical representation.
It accepts a wrong word (foxs).

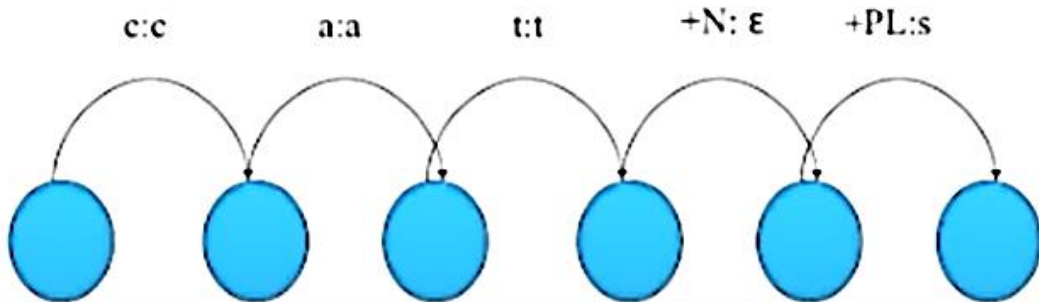
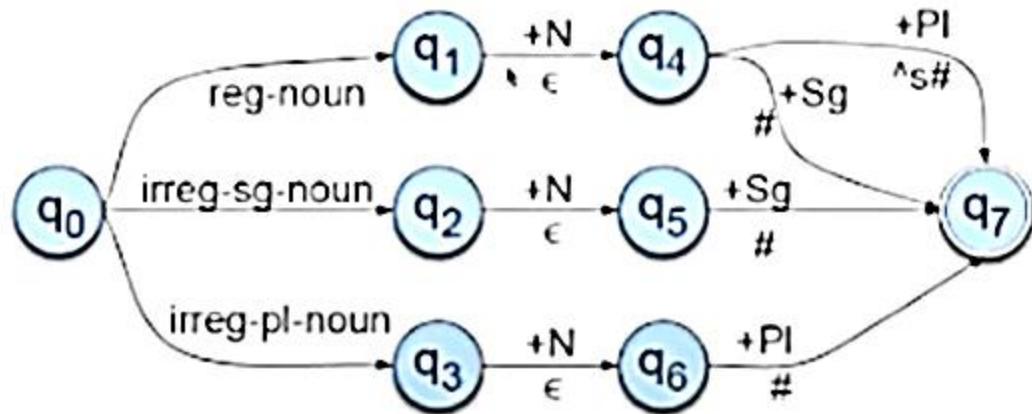
A FST for Simple English Nominals

Regular Singular noun
cat → cat

Regular Plural noun
cat → cats

Irregular singular noun
ox → ox

Irregular plural noun
ox → oxen



Applications of FST in NLP

- Spell checking and correction
- Grammar checking
- Text normalization
- Morphological Analysis
- Part of Speech tagging

End of chapter 3